Routes to Safety

An evacuation-services map from open geospatial data — datasets, tiles, routing, rendering

DuckDB · Overture Maps · Planetiler · PMTiles · MapLibre GL · OSRM

Vishal Kumar · Shaishav Maisuria

FOSS4G NA 2026 · Sacramento · Mon 2 Nov, 09:00–12:00 · Room Beavis · github.com/vishal4c/routes-to-safety-workshop

Open geospatial data in. An evacuation-services map — and the routes to it — out.

Hands-on with open datasets (OpenStreetMap, Overture Maps themes, CAL FIRE) → vector tiles → a routing graph for route-based analysis → rendered in the browser. Four open tools, one per phase.

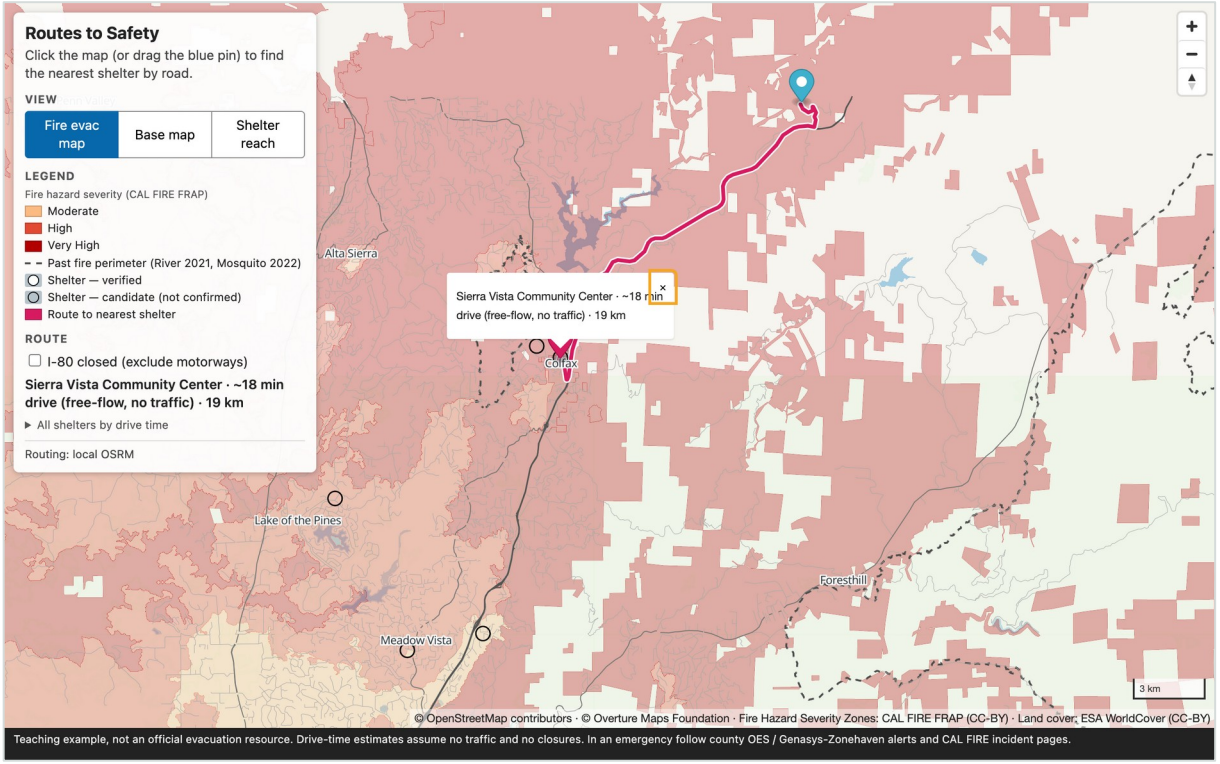
Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

What we work with — open datasets and open tools

Open dataset	What we take from it	Licence
OpenStreetMap	the road network OSRM routes on — one .osm.pbf extract (Geofabrik), turn restrictions, speeds	ODbL
Overture Maps	themes transportation · places · divisions · base · buildings ⟶ roads, POIs, towns, water, land cover, 3D buildings, as GeoJSON	ODbL · CDLA-P 2.0
CAL FIRE FRAP	Fire Hazard Severity Zones · fire perimeters (ArcGIS feature services)	CC-BY
ESA WorldCover	land cover 10 m (via Overture base)	CC-BY 4.0

Open tool	Phase	What it does for us
DuckDB + spatial	1	SQL over GeoJSON, and over Overture's Parquet on S3 — a county out of the planet in seconds (bbox pushdown)
Planetiler ⟶ PMTiles	2a	GeoJSON ⟶ vector tiles ⟶ one static file; no tile server, Range requests only
OSRM	2b · 2c · 3c	routing graph from the OSM extract; /nearest · /table · /route ⟶ nearest by road, reach bands, closures
MapLibre GL JS	3	renders the tiles from a JSON style, three views over the same file, routing wired into the page

What you will have built



Public demo: vishal4c.github.io/routes-to-safety-workshop/demo/ — Placer County I-80 corridor: Rocklin → Auburn → Colfax → Dutch Flat / Foresthill

Your own vector tiles

roads · water · hazard zones · fire perimeters · city labels · shelters — one 6.5 MB PMTiles file

Three views, one at a time

Fire evac map · Base map · Shelter reach — the legend follows the view, nothing is overlaid

Live routing in every view

click anywhere → nearest shelter by road, route line, ranked list (OSRM)

'I-80 is closed'

one parameter, new answer — and why that is not the same as detection

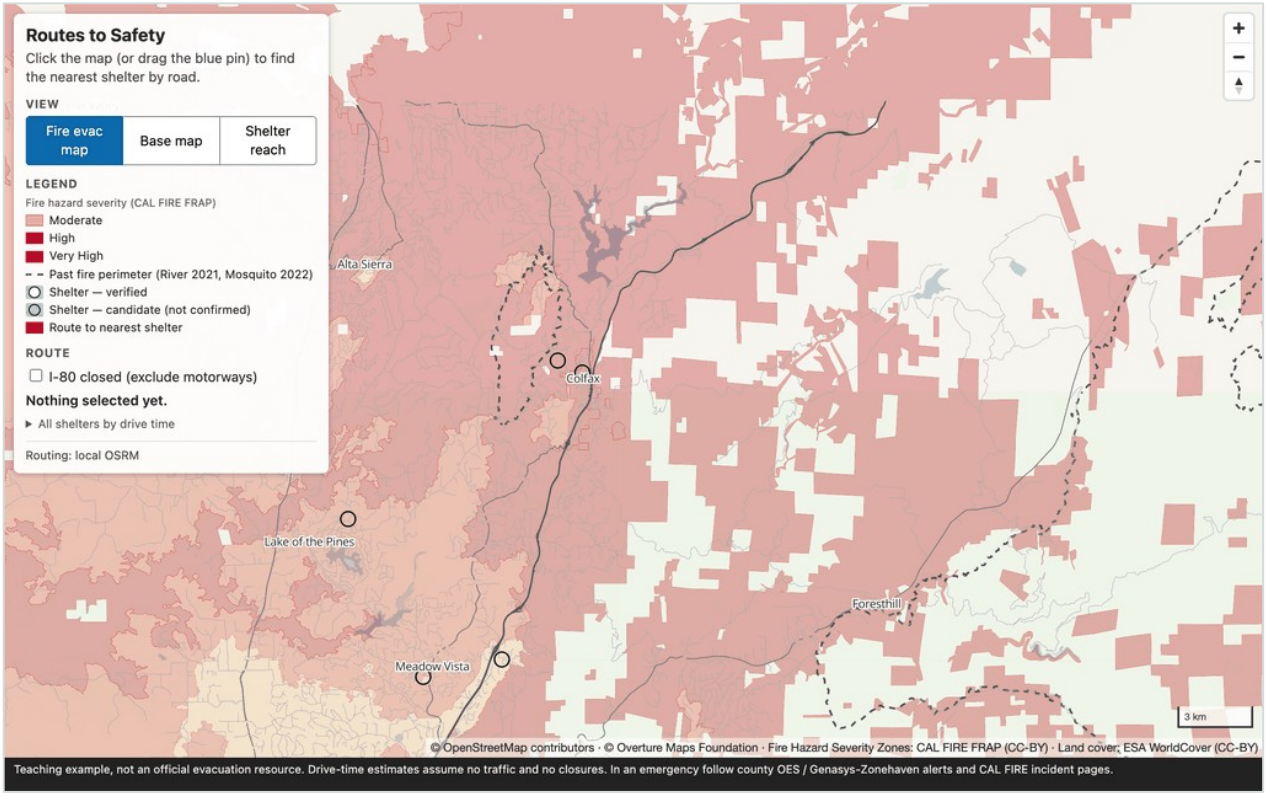
Shelter reach

≤ 10 min green · ≤ 20 blue · ≤ 30 light red, from one OSRM /table call

Static files only

no database, no tile server; works offline in this room

The app in motion



Animated in slideshow mode. Recorded against the local OSRM container; the public demo uses router.project-osrm.org (no closure toggle there).

1 Drop a pin at Dutch Flat → Sierra Vista CC, 18 min

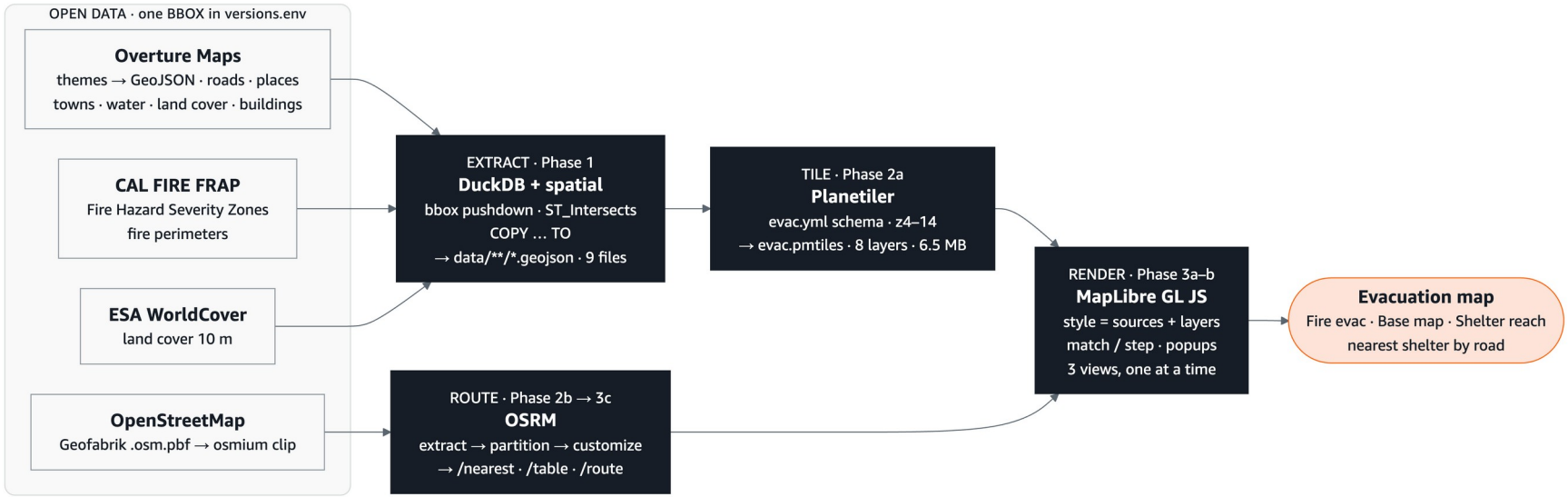
2 Tick "I-80 closed" → same shelter, 31 min via Dutch Flat Rd

3 Foresthill: nearest on the map (Georgetown, 12 km) ≠ nearest by road (Placer High, 36 min)

4 Shelter reach view → 10 / 20 / 30-min bands; click Placer High → only its reach

5 Base map view → a plain street map; the pin and the route stay — routing works in every view

The pipeline: one extract ⟶ a basemap and live routes



Phase 4: change one bbox line in versions.env ⟶ **the nine Phase 4 steps** — fetch-overture.sh · fetch-osm.sh ⟶ build-osrm.sh · build.sh · make-coverage.py — rebuild everything for your own town. Everything is pinned (DuckDB 2.0.0 · Planetiler 0.10.2 · pmtiles 1.31.2 · MapLibre 6.14 · OSRM v26.10.0) and verified by CHECKSUMS.sha256.

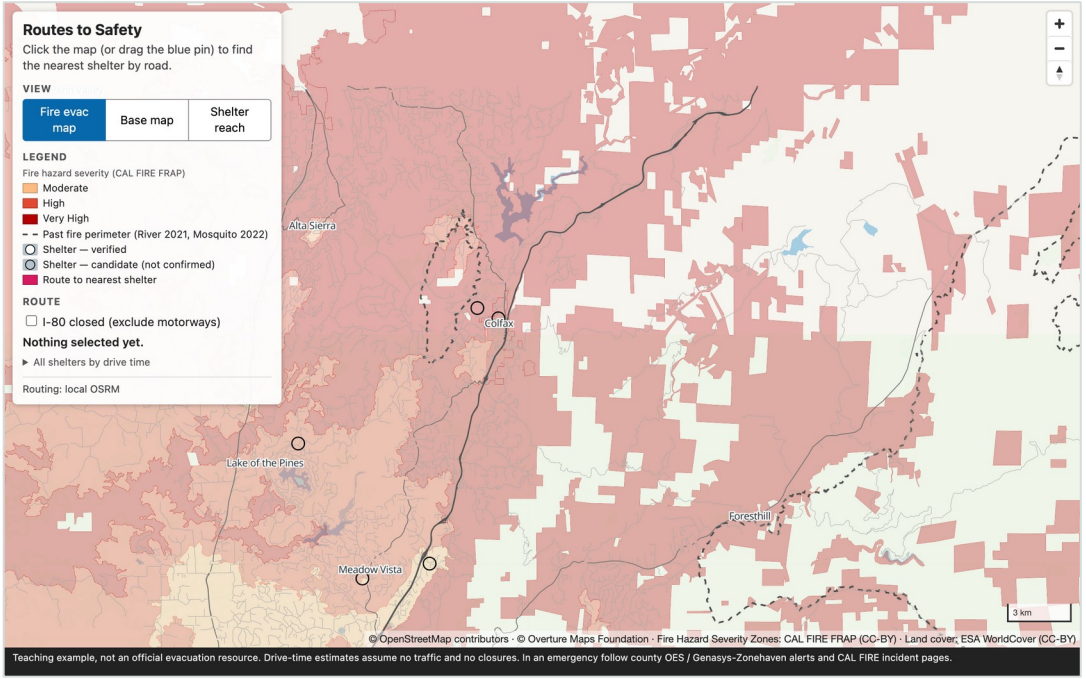
Agenda — five phases, one map

Phase	You do	Tools
0 · Setup	pick Codespaces / Docker / native · checkpoint line — the same green line on every screen	DuckDB
1 · Data prep	1a explore · 1b shelters × hazard · 1c export	DuckDB · Overture
Break		
2 · Map artifacts	2a tiles · 2b routing graph · 2c reach bands — the three artifacts the app needs (2b, 2c prebuilt)	Planetiler · PMTiles · OSRM
3 · Application	3a style · 3b three views · 3c routing (+ the I-80 closure) · 3d finished app	MapLibre · OSRM
4 · Adapt	one box in versions.env ⟶ the same nine steps for your town; swap the hazard	versions.env · scripts/
Done	Questions in the hallway	

Sticky note on your lid: **green** = fine · **red** = stuck, come find me. Finish early ⟶ every phase card has a finish-early exercise.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Study area: the Placer County I-80 corridor



Rocklin → Auburn → Colfax → Dutch Flat / Foresthill

bbox −121.40, 38.72, −120.52, 39.20 (W, S, E, N) — about 0.4 square degrees, 36,294 drivable road segments, 10 shelters

Why here

River Fire, Aug 2021 — Colfax evacuated for ~2 days, 2,619 acres on the corridor

Mosquito Fire, Sep 2022 — 76,771 acres; the shelter moved mid-incident to Sierra College, Rocklin. The safe place is not a fixed point.

One interstate, steep canyons: nearest on the map is often not nearest by road

What we leave out

shelter status (open/closed/full) · drive time ≠ evacuation time · live closures · capacity

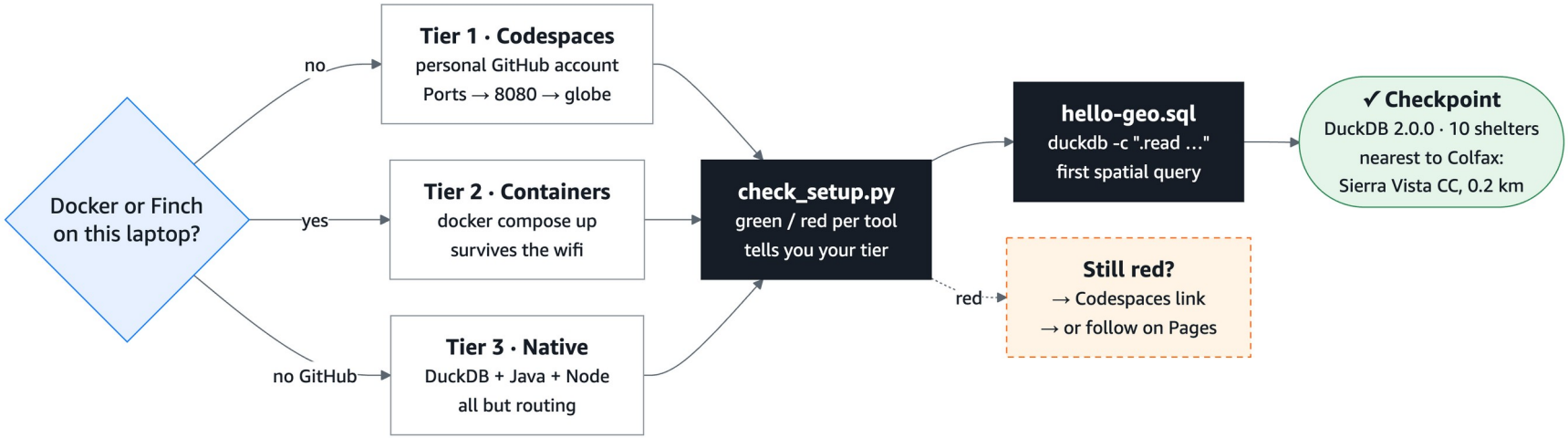
Shelters: 1 verified (Sierra College) + 9 candidates from Overture places; hazard: CAL FIRE FHSZ 2024 (SRA + LRA). Teaching example, not an official evacuation resource. Drive-time estimates assume no traffic and no closures.

PHASE 0

Setup

Welcome · pick a way to run · setup checkpoint DuckDB 2.0.0 · Codespaces / Docker / native

Phase 0 in one picture — pick a way to run, run two lines, hands up



Checkpoint: OK — DuckDB 2.0.0 · 10 shelters · nearest to Colfax: Sierra Vista Community Center, 0.2 km

Three ways to run — pick one, keep a neighbour

Tier	Who	Routing	Notes
1 · Codespaces	no Docker on the laptop	yes	**personal** GitHub account · sit next to a Docker user · 8080 via Ports tab → globe
2 · Local containers	**recommended** if you have Docker / Finch	yes	survives wifi loss · Finch for corporate laptops: finch compose up
3 · Native	DuckDB + Java + Node, no containers	no	Windows: Git Bash · map + shelters only (OSRM ships no native Win/Mac binary)
4 · Browser only	laptop failed	no	Pages site: follow along, demo still routes via the public OSRM server

PHASE 0 · setup checkpoint

Run both lines from the repo root. Hands up when your line matches the one on the right.

Still red? Open the Codespaces link. Codespaces blocked? Open the Pages link and follow along.

```
python3 setup/check_setup.py
duckdb -c ".read setup/hello-geo.sql"
```

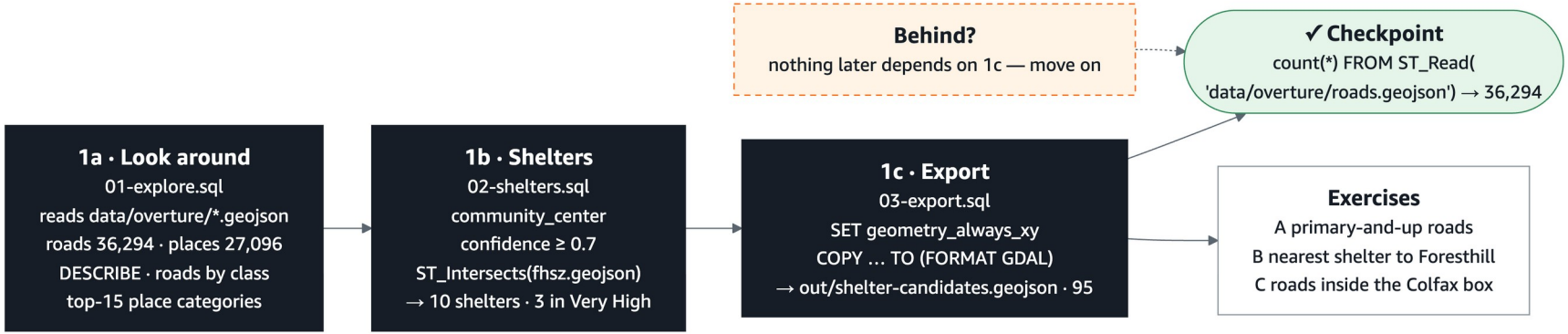
```
OK — DuckDB 2.0.0 · 10 shelters (1 verified, 9 candidate)
  · nearest to Colfax (crow-flies): Sierra Vista Community Center, 0.2 km
```

PHASE 1

Data prep with DuckDB

1a look around · 1b candidate shelters · 1c export · exercises DuckDB 2.0.0 + spatial · Overture Maps
2026-09-23.1

Phase 1 in one picture — three SQL files, one folder of GeoJSON



Phase 1: 1a explore · 1b shelters · 1c export · **Checkpoint** SELECT count(*) FROM ST_Read('data/overture/roads.geojson') → 36,294

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Overture Maps in one slide

Open map data from the Linux Foundation, released monthly as **GeoParquet** in themes: transportation · places · base · buildings · divisions

Built from OpenStreetMap plus other sources; licence differs by theme (ODbL / CDLA-Permissive 2.0)

Parquet stores a bbox struct per row → DuckDB skips every file and row group outside your rectangle. Planet → Placer corridor without downloading the planet; we stage the result as GeoJSON in data/overture/.

Attribution on every map you publish: © OpenStreetMap contributors · © Overture Maps Foundation

36,294 drivable roads	**27,096** places
121 towns (divisions)	**1,920** water polygons
2,564 land-cover polygons	**12,606** buildings (Auburn)

Our study area, release 2026-09-23.1 — fetched to GeoJSON by scripts/fetch-overture.sh in 24 s

SQL in three lines, then one spatial predicate

```
LOAD spatial;
SELECT class, count(*) AS n      -- what do I want back?
FROM ST_Read('data/overture/roads.geojson')
GROUP BY class ORDER BY n DESC;  -- one row per group
```

No GIS yet — this is just a table.

SELECT what · WHERE which rows · GROUP BY count per group

Three results you will see: residential 25,995 · tertiary 4,333 · secondary 2,792 · primary 1,305 · motorway 541 · trunk 63

```
LOAD spatial;
SELECT s.name FROM shelters s, fhsz h
WHERE ST_Intersects(s.geom, h.geom)   -- yes / no
  AND h.FHSZ_Description = 'Very High';
```

A predicate is a yes/no question about two shapes.

It goes in WHERE like any other condition.

Phase 1b answer: three shelters sit in a Very High zone — Colfax High School, Sierra Vista Community Center, Georgetown School.

DuckDB 2.0 gotcha: SET geometry_always_xy = true; right after LOAD spatial; or the GeoJSON writer swaps lat/lon.

Columns cheat sheet — raw Overture vs. our GeoJSON

You want	Raw Overture (S3)	Our data/overture/*.geojson
a place's name	names.primary	name
a place's category	taxonomy.primary	category
how sure Overture is	confidence (use >= 0.7)	same
road type	class, subclass	same
the shape	geometry	same

scripts/fetch-overture.sh already flattened these — today you type the right-hand column.

categories was removed in Sep 2026 → taxonomy.primary

There is no "fairground" category — Gold Country Fairgrounds hides under another one.

Writing GeoJSON: SET geometry_always_xy = true first, or lat/lon come out swapped.

The noise is the lesson: John L Sullivan Chevrolet is a community_center at confidence 0.92.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

PHASE 1 — hands-on: extract the data (working dir = repo root, out/ exists)

1a · Look around

duckdb -c ".read 00-data/01-explore.sql"

DESCRIBE roads · roads by class · top-15 place categories ⟶ expected output printed in the file

1b · Candidate shelters

duckdb -c ".read 00-data/02-shelters.sql"

community_center + confidence ≥ 0.7 ⟶ spot the Chevrolet dealer ⟶ open data/shelters.geojson ⟶ ST_Intersects with FHSZ

1c · Export

write GeoJSON with SQL (right):

duckdb -c ".read 00-data/03-export.sql" ⟶ out/shelter-candidates.geojson, 95 places

Nothing later depends on 1c — behind? move on.

```
LOAD spatial; SET geometry_always_xy = true;
COPY (SELECT id, name, category, confidence, geom
      FROM ST_Read('data/overture/places.geojson')
      WHERE category IN ('school','community_center', … ))
TO 'out/shelter-candidates.geojson' (FORMAT GDAL, DRIVER 'GeoJSON');
```

Exercises

A count motorway + trunk + primary ⟶ 1,909

B crow-flies nearest shelter to Foresthill (–120.8182, 39.0199) ⟶ Georgetown School, 12.4 km. **Remember it for Phase 3c.**

C (finish early) count the roads inside the Colfax box with ST_Intersects

Checkpoint duckdb -c "LOAD spatial; SELECT count(*) FROM ST_Read('data/overture/roads.geojson')" ⟶ **36,294**

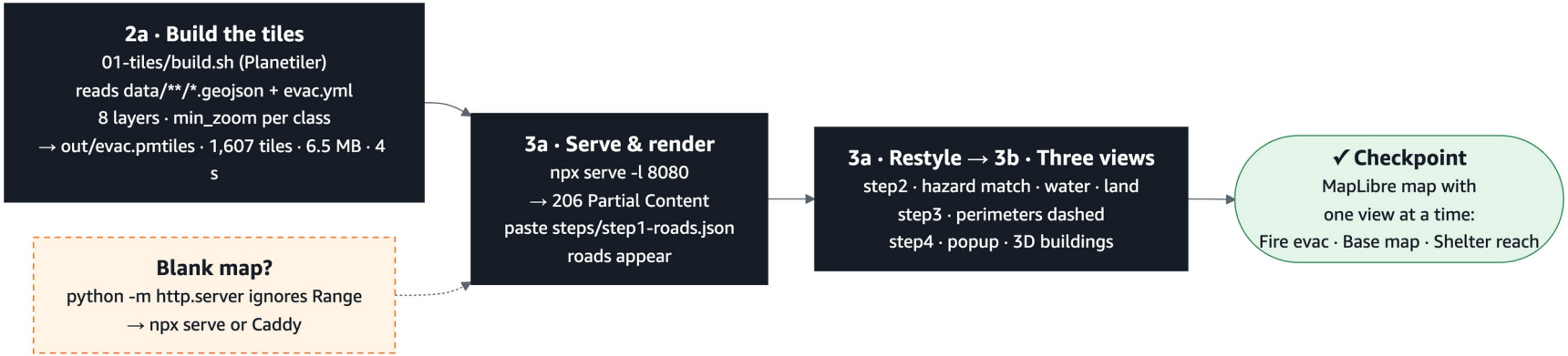
PHASE 2 → 3

Map artifacts → the application

2a build the tiles · 3a serve & style · 3b three views Planetiler 0.10.2 · go-pmtiles 1.31.2 · MapLibre GL JS 6.14 · pmtiles.js 4.5

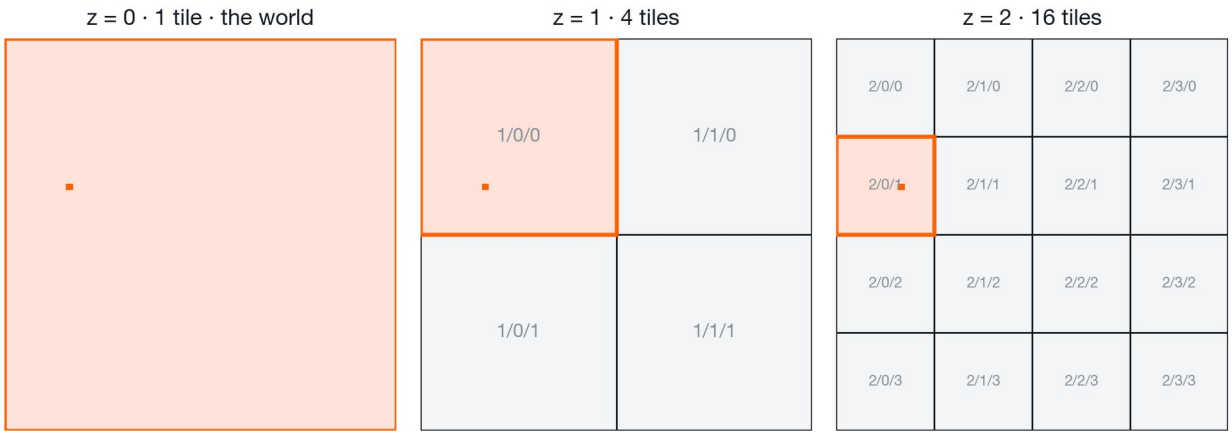
Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 2 ⟶ 3 in one picture — GeoJSON ⟶ one tile file ⟶ a styled map



Phase 2a ⟶ 3b: 2a build the tiles · 3a serve, render, restyle · 3b three views · **Checkpoint** curl -sI -H "Range: bytes=0-15" .../out/evac.pmtiles ⟶ 206 Partial Content

Why tiles: the map is cut into a z/x/y quadtree



z	tiles on Earth	over our bbox
4	256	1
8	65,536	4
10	1,048,576	12
12	16.8 M	88
14	268 M	1,218

Each zoom splits every tile into four (a quadtree). A tile is addressed z/x/y and holds only what falls inside it — the browser fetches the ~12 tiles on screen, never the whole dataset. Our file: 1,607 tiles, z4–14, 6.5 MB, one file.

Each zoom level splits every tile into four. z0 is the whole world in one tile; by z14 the planet is 268 million tiles — but our rectangle touches only 1,218 of them. The browser fetches the handful under the viewport, at the zoom it is showing, and nothing else.

That is why one 6.5 MB file replaces a 17 MB GeoJSON and a database: 1,607 tiles for z4–14, each holding only the geometry that matters at that zoom (motorways from z6, residential streets from z10). Zoom in → a new, smaller square; pan → the neighbour square.

Vector tiles, PMTiles and the Range-request trap

A browser cannot draw 36,294 roads from a 17 MB GeoJSON fast.

So we cut the map into a z/x/y grid. Each tile holds geometry + attributes, not pixels ⟶ the browser draws it, so you restyle without rebuilding.

PMTiles = every tile in one file with an index. The browser asks for byte ranges; no tile server, works from any static host (GitHub Pages, S3, a USB stick).

The trap: python -m http.server ignores Range headers ⟶ the map stays blank. Use Caddy (:8080) or npx serve -l 8080 . and look for **206 Partial Content**.

```
# the only test you need
curl -sI -H "Range: bytes=0-15" http://localhost:8080/out/evac.pmtiles | head –1
HTTP/1.1 206 Partial Content
```

```
# 01-tiles/evac.yml  (excerpt — 8 layers, all local files, zero network)
sources:
  roads: { type: geojson, local_path: data/overture/roads.geojson }
  towns: { type: geojson, local_path: data/overture/towns.geojson }
layers:
  - id: roads
    features:
      - { source: roads, geometry: line, min_zoom: 6,
          include_when: { class: [motorway, trunk, primary, secondary] } }
      - { source: roads, geometry: line, min_zoom: 10, min_size: 1,
          exclude_when: { class: [motorway, trunk, primary, secondary] } }
```

MapLibre style: sources, layers, and two expressions

```
"fill-color": ["match", ["get", "FHSZ_Description"],
  "Moderate",  "#fdbb84",
  "High",      "#e34a33",
  "Very High", "#b30 000",
  "#cccccc"]                    // fallback
```

```
"line-width": ["step", ["zoom"], 0.6, 10, 1.2, 13, 1.6]
```

match — one output per category (hazard class ⟶ OrRd)

step — one output per range (zoom ⟶ width)

Case and spelling must match exactly: "Very High", not "very_high". FHSZ is also there as 1 | 2 | 3.



A style is JSON: sources (our one PMTiles file) + layers (one per thing you draw, bottom to top).

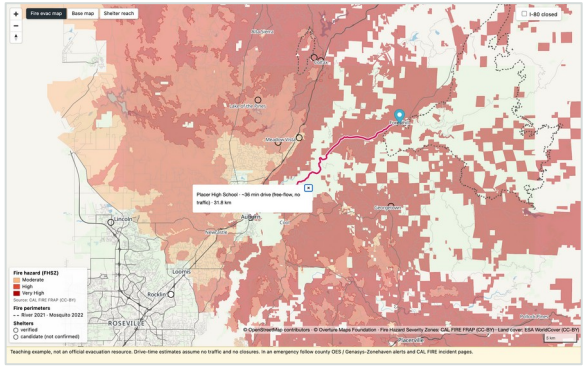
Paste-in steps: index.html sorts your pasted layers by a fixed order and de-duplicates by id, so paste order never matters and step 3 can be pasted over steps 1–2.

Labels: a symbol layer with text-field + text-font; glyph PBFs are vendored in 02-web/fonts so labels work offline.

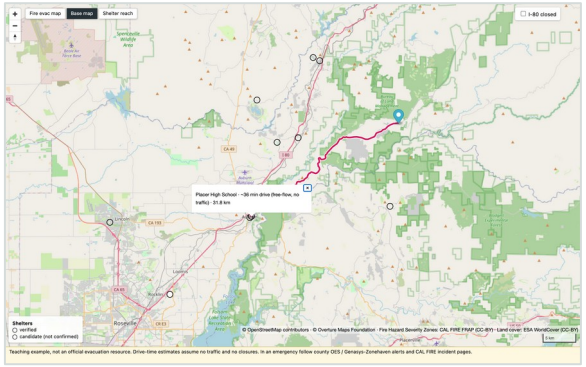
Three views, one at a time: Fire evac map · Base map (Esri street raster) · Shelter reach — setLayoutProperty(visibility) over a list of ids, legend swapped with it; no style reload, so routing keeps working.

Shelter reach: a GeoJSON source + setFilter(shelter_id); the 0–10 / 10–20 / 20–30-min rings never overlap, so they stay translucent over the roads without blending. Precomputed with OSRM /table in Phase 2c.

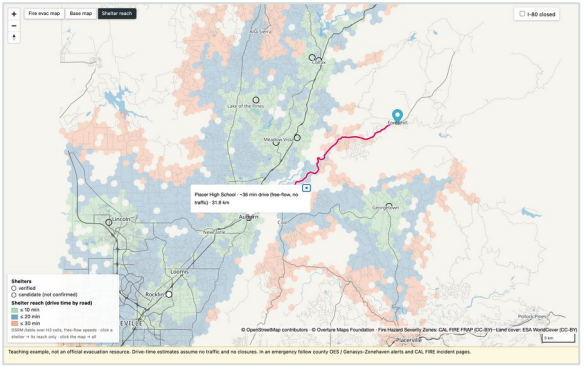
What Phase 3 produces: three views of the same tiles



Fire evac map — our tiles: FHSZ Moderate / High / Very High, dashed perimeters, city labels, shelters, route



Base map — a standard street raster (Esri, no API key); only shelters, the pin and the route on top (needs internet)



Shelter reach — ≤ 10 min green · ≤ 20 blue · ≤ 30 light red from every shelter; click a shelter for its own bands

Strictly one view at a time: Fire evac map, Base map or Shelter reach — the legend follows the view; the reach bands are never drawn on top of the hazard fill. **Phase 3a checkpoint:** Very High solid red · water blue · 10 shelter circles (1 filled) · Mosquito perimeter dashed · city labels · attribution visible. Hazard edges are zone boundaries, not a guarantee: a road just outside a polygon is not "safe".

PHASE 2a ⟶ 3b — hands-on: build and style the map

Open the page: Codespaces ⟶ Ports tab ⟶ 8080 ⟶ globe · Docker ⟶ http://localhost:8080/02-web/ · Native ⟶ npx serve -l 8080 . then the same URL

2a · Build the tiles

01-tiles/build.sh # = java -jar planetiler.jar generate-custom --schema=01-tiles/evac.yml --output=out/evac.pmtiles

pmtiles show out/evac.pmtiles

Expected: 1,607 tiles, zoom 4–14, 8 layers, ≈ 6.5 MB, ~4 s. · no PMTiles? ⟶

cp data/prebuilt/evac.pmtiles out/

3a · Serve & render (step 1)

in 02-web/index.html find

// STEP 1: paste here ⟶ paste steps/step1-roads.json ⟶ reload; uncomment AttributionControl ⟶ grey roads + attribution bottom-right

3a · Restyle (steps 2–4)

step2: hazard match (OrRd) · water fill · land cover · shelters filled = verified, hollow = candidate · city labels

step3 (if time): fire perimeters dashed grey

step4 (finish early): click popup with name · fill-extrusion on buildings (zoom ≥ 13, tilt)

stuck? ⟶ paste steps/step3-styled.json; totally lost ⟶ open 02-web/solution.html

Checkpoint Very High solid red · water blue · 10 shelters (1 filled) · Mosquito perimeter dashed · attribution visible

3b · Three views — try the Base map and Shelter reach buttons, one view at a time: a plain street map, or each shelter's 10 / 20 / 30-min drive-time bands (click a shelter to isolate it); then come back, it is the same page.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

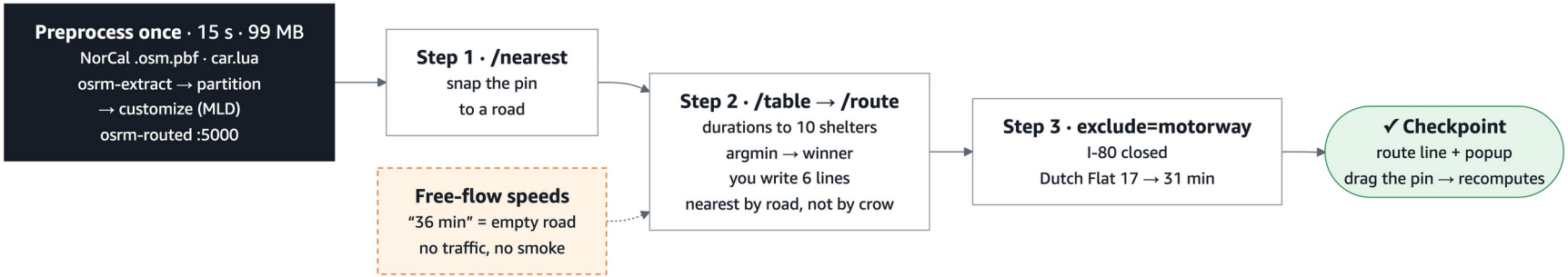
PHASE 3c

Routing in the application (OSRM)

the routing graph (2b) · /nearest · /table → argmin → /route · I-80 closed OSRM v26.10.0 (MLD) · tiers 1–2 route, tiers 3–4 pair up

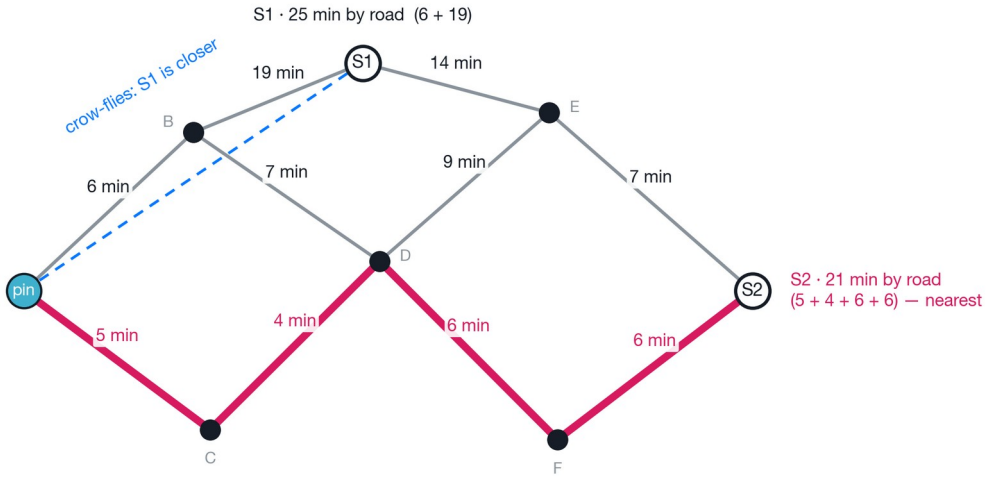
Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 3c in one picture — preprocess once, then three HTTP calls



Phase 3c: hello route (/nearest) · nearest by road (/table ⟶ argmin ⟶ /route) · I-80 closed (exclude=motorway) · **Teaching numbers** Dutch Flat ⟶ Sierra Vista CC 17.1 min, 31.3 min with I-80 closed · Foresthill ⟶ Placer High 35.8 min / 31.8 km

Why a routing graph: nearest on the map ≠ nearest by road



Nodes = intersections · edges = road segments · weight = seconds at free-flow speed. Dijkstra grows the cheapest frontier from the pin until it touches a shelter. OSRM precomputes this once (extract → partition → customize, 15 s, 99 MB) so every /route or /table answer is milliseconds.

The crow lies

S1 is closest on the map, 25 min by road. S2 looks farther but is 21 min: a canyon, a river or a missing bridge bends the real distance.

A road network is a graph

nodes = intersections, edges = road segments weighted in seconds. Dijkstra grows outward from the pin, cheapest node first, until it has settled every shelter.

Why preprocess

on 36,294 roads a search per click is fine; on a region it is not. OSRM builds the graph once (15 s, 99 MB) and answers /nearest, /table and /route in milliseconds.

Build-once artifact

like the tiles: made in Phase 2, consumed by the app in Phase 3c.

Edge weights = length ÷ free-flow speed. Pink: the cheapest path pin → C → D → F → S2. Grey dashed: the straight line the map suggests.

Routing in three minutes, and three HTTP calls

A road map is a graph: intersections are nodes, segments are edges, each edge costs seconds (length ÷ free-flow speed).

A route is the cheapest path — Dijkstra explores outward until it reaches the target.

Too slow at scale → OSRM preprocesses once: **MLD** partitions the graph into cells and precomputes costs between cell borders. Our graph: 15 s to build, 99 MB.

Weights are free-flow speeds, not traffic. "36 minutes" means 36 minutes on an empty road.

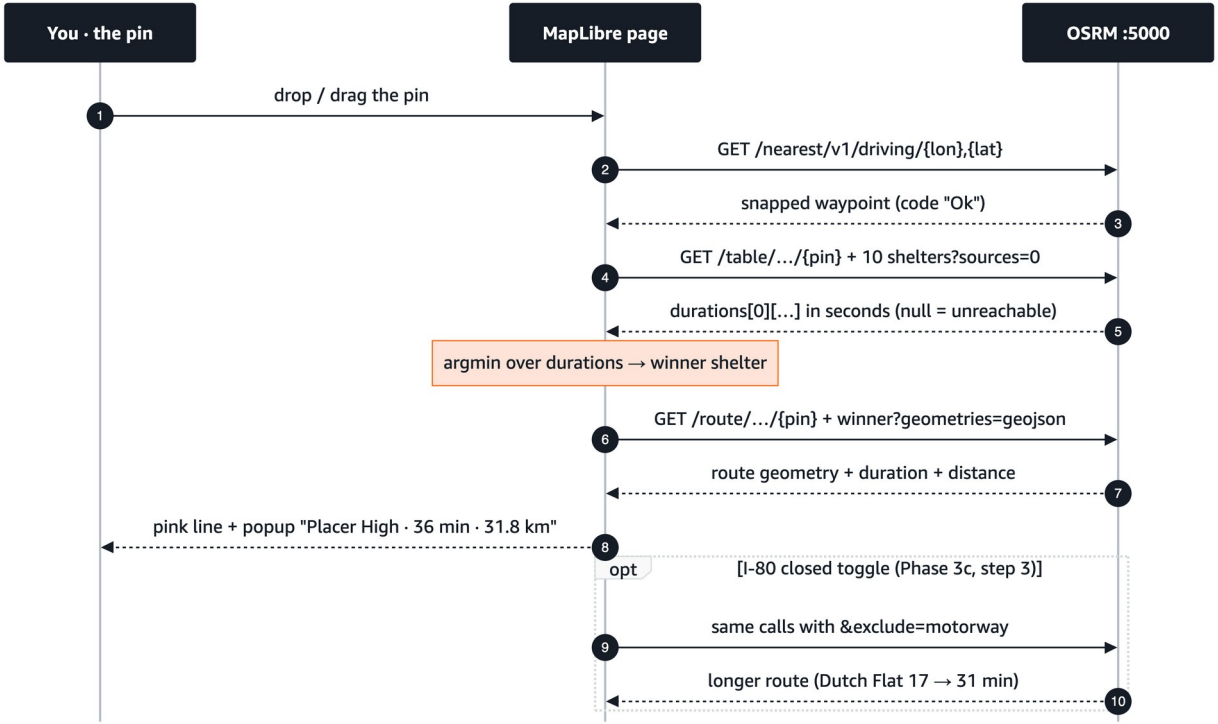
Call	Question it answers
/nearest	Which road point is closest to this coordinate? (snap)
/route	Path + duration + distance between two points
/table	Durations (and distances) from many points to many points

```
GET $OSRM_URL/{service}/v1/driving/{lon},{lat};{lon},{lat}   # longitude first!
```

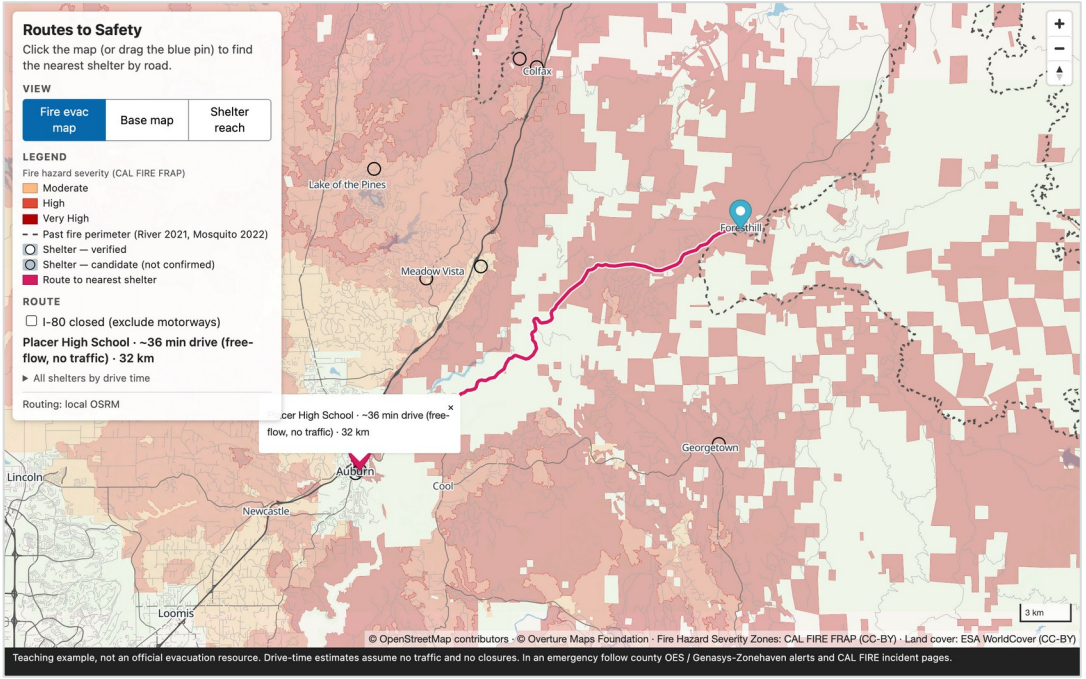
Tier	$OSRM_URL
1 Codespaces	<forwarded-8080-URL>/osrm (Ports → globe; never forward 5000)
2 Local Docker / Finch	http://localhost:8080/osrm (Caddy strips /osrm → osrm:5000)
3 Native · 4 Browser	no routing — pair with a neighbour

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

What the browser and OSRM say to each other



Nearest on the map ≠ nearest by road



Foresthill sits on a ridge above the American River canyon.

Exercise B (crow-flies): Georgetown School, 12.4 km

Phase 3c (/table, by road): Placer High School, 35.8 min / 31.8 km. Georgetown is 59.7 min / 54 km by road.

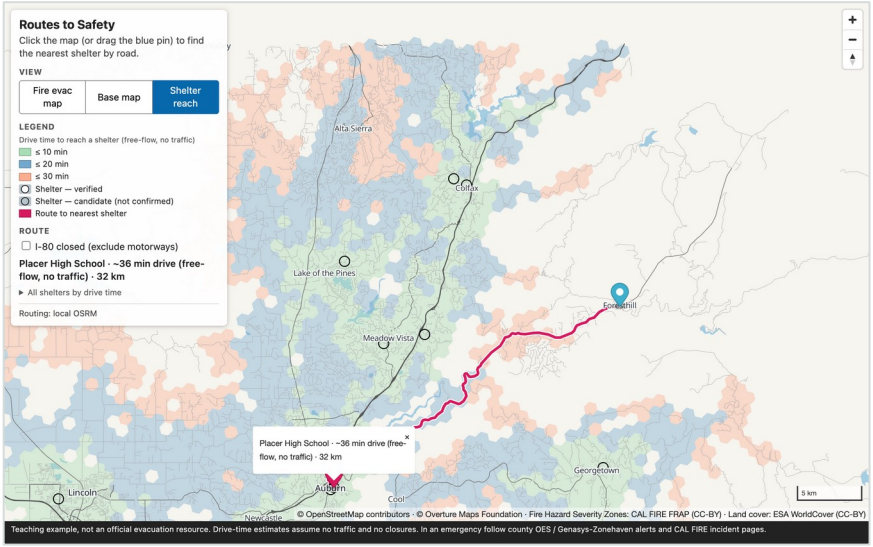
A ridge, a river canyon or a missing bridge makes the "close" shelter far. This is why the shelter list is ranked by /table, not by ST_Distance.

How the graph was built: osmium extract -b <bbox> norcal-latest.osm.pbf → osrm-extract -p car.lua → osrm-partition → osrm-customize → osrm-routed --algorithm mld --max-table-size 500

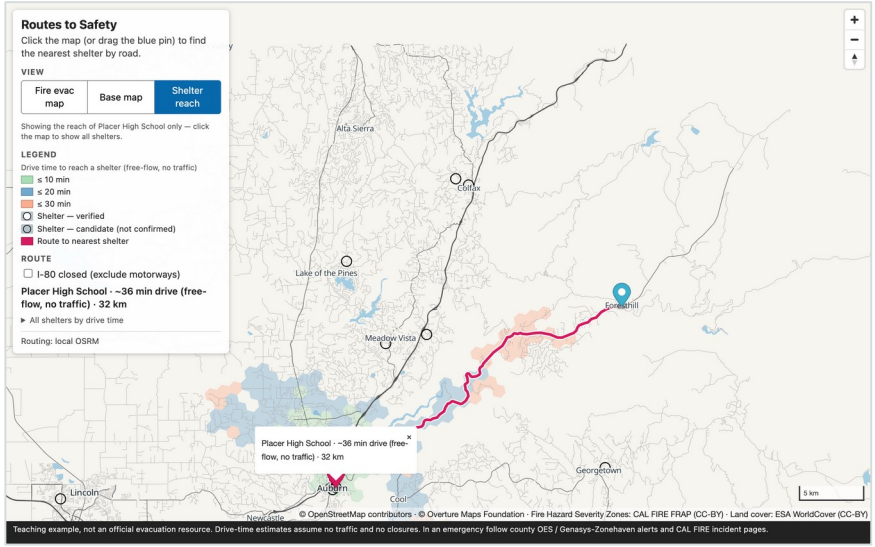
Finish early? build the graph yourself: scripts/build-osrm.sh data/osm/colfax.osm.pbf

Demo at Foresthill (−120.8182, 39.0199): ranked list from one /table call, route from /route.

From routes to coverage: /table once, draw the reach



Shelter reach view — all shelters: ≤ 10 min green · ≤ 20 blue · ≤ 30 light red (dissolved H3 cells)



Click a shelter ⟶ only its 10 / 20 / 30-minute reach

scripts/make-coverage.py: H3 res-8 grid over the bbox (3,341 cells with a road) ⟶ one batched /table call (cells × 10 shelters) ⟶ DuckDB dissolves the cells into one band per shelter. Stretch A in Phase 3c is the 400-point version with grid.sql.

PHASE 3c — hands-on: routing (working dir = repo root)

Step 1 · Hello route

curl "$OSRM_URL/nearest/v1/driving/-121.08,38.90" (PowerShell: curl.exe)

Expected: JSON with "code":"Ok" and a waypoints array. Then /route…? geometries=geojson; paste via steps/step1-route.js → pink line Foresthill → Placer High, console "Route: 35.8 min, 31.8 km".

Step 2 · Nearest shelter by road

in steps/step2-scaffold.js

you write 6 lines: /table URL → argmin durations[0] → /route → setData → popup. Compare with Exercise B. Drag either marker.

not routing? → open steps/step3-solution.js

Step 3 · I-80 is closed

add the

&exclude=motorway toggle; start from **Dutch Flat (-120.838, 39.198)**: 17 → 31 min to Sierra Vista CC, the route leaves I-80 for Dutch Flat Rd / Rollins Lake Rd.

Real closures aren't a parameter — nobody flags a downed power line before you drive into it. This shows the calculation, not detection.

Tiers 3/4: no routing by design — pair with a Docker/Codespaces neighbour or follow the screen.

Finish early? Stretch A 15-min reach: grid.sql → one /table POST (400 points) **Stretch B** build the graph for Colfax — finish at home, the repo has the solution.

Checkpoint drag anywhere → line + popup update; toggle changes the route; footer disclaimer visible.

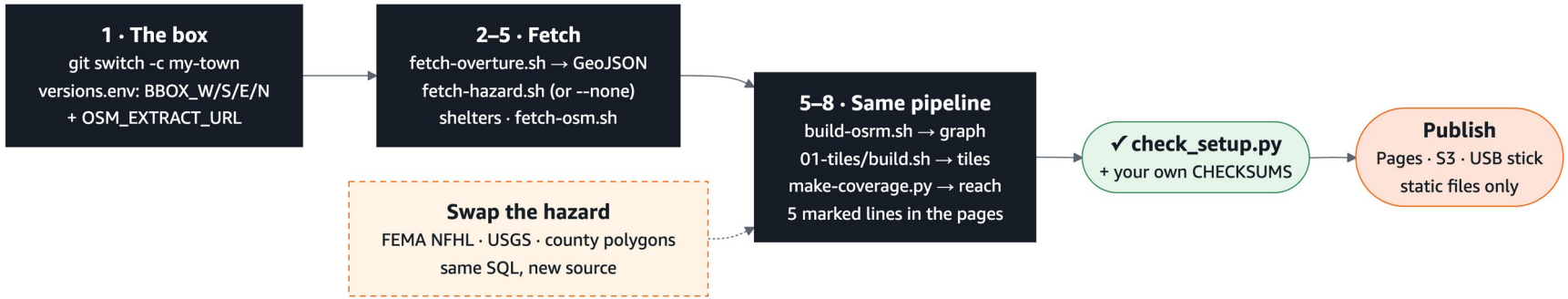
PHASE 4

Adapt the pipeline to your community

one box → fetch → build → the same pages versions.env · fetch-overture.sh · fetch-osm.sh · build-osrm.sh · build.sh · make-coverage.py

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Phase 4 in one picture — one box, the same scripts, your town



Phase 4: fetch-overture.sh ⟶ fetch-hazard.sh ⟶ shelters ⟶ fetch-osm.sh + build-osrm.sh ⟶ build.sh ⟶ make-coverage.py ⟶ 5 marked lines ·
Pinned: DuckDB 2.0.0 · Planetiler 0.10.2 · pmtiles 1.31.2 · MapLibre 6.14 · OSRM v26.10.0

Reproducibility, then swap the community

If we can't rebuild it next year with the same answers, it's a demo, not a method.

versions.env — one file pins every tool and image; scripts/check-versions.sh fails CI on drift

Dated extracts: Overture 2026-09-23.1, a dated Geofabrik norcal snapshot (2026-10-22)

data/CHECKSUMS.sha256 — verify the 143 MB bundle (OSM clip, OSRM graph, tiles, planetiler.jar) you downloaded

One script per step: fetch-overture.sh · fetch-hazard.sh · fetch-osm.sh ⟶ build-osrm.sh · 01-tiles/build.sh · make-coverage.py — all GeoJSON in, artifacts out

PHASE 4 · Swap the community

1. git switch -c my-town · versions.env: BBOX_W/S/E/N + OSM_EXTRACT_URL
2. scripts/fetch-overture.sh (≈ 20 s) · scripts/fetch-hazard.sh --none
3. shelters ⟶ fetch-osm.sh + build-osrm.sh ⟶ build.sh ⟶ make-coverage.py

The OSM region download is the big one (a US state, 100 MB – 1 GB); everything else runs in seconds.

Swap the hazard

FEMA NFHL flood zones instead of fire zones: polygons ⟶ ST_Intersects ⟶ tiles ⟶ match on a zone field. A data change, not a code change.

Going live (CAL FIRE perimeters feed) **raises the stakes on every caveat** from "what we leave out": status, drive time, closures, capacity.

Laptop ⟶ planet: same shape, bigger inputs

Stage	Today (0.4 sq°)	At scale
Extract	DuckDB bbox pushdown, 24 s, six GeoJSON files (38 MB)	same SQL; a region at a time; run where the data lives
Tiles	Planetiler 4 s ⟶ 6.5 MB PMTiles	planet-scale builds are a documented Planetiler use case (bigger heap, more disk)
Serve	npx serve / Caddy on :8080	PMTiles on object storage behind a CDN — one file, Range requests, no tile server
Routing	OSRM MLD graph 15 s, 99 MB	one regional graph per region, not one planet graph per laptop; Valhalla / GraphHopper for isochrones, traffic
Freshness	pinned release 2026-09-23.1	Overture publishes monthly — bump the pin in versions.env, rerun, diff the answers

In the wild: OSM underlies OSRM (public server router.project-osrm.org) · Planetiler builds OpenMapTiles-style and custom tiles · PMTiles is the Protomaps single-file format · Overture releases under the Linux Foundation · MapLibre is the open fork lineage of Mapbox GL. Real systems add live closures, shelter status, capacity and traffic. We didn't.

WRAP-UP

Recap, licences, feedback

What you built · where it lives · how to ask for help

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0

Recap — the pipeline you built



Open data ✦ open tools: no accounts, no licences to buy, no server to rent — you can re-run all of it on your own laptop.

Two questions, one extract: where are the safe places, and how do people reach them in time — a basemap and routes from the same data.

It answers them on a good day. It cannot tell you if today is a good day: shelter status, traffic, live closures and capacity are not in it.

Keep: the repo (MIT code, CC BY-SA 4.0 materials), the 143 MB data bundle (release data-v2), the public demo, and the habit: one box, one script per step.

Licences, attribution, links

What	Licence
Code	MIT
Materials (slides, docs)	CC BY-SA 4.0
OpenStreetMap	ODbL
Overture Maps	ODbL / CDLA-Permissive 2.0 (by theme)
CAL FIRE FHSZ, perimeters	CC-BY
ESA WorldCover	CC-BY 4.0
Noto Sans glyphs	SIL OFL 1.1

Repo github.com/vishal4c/routes-to-safety-workshop

Demo vishal4c.github.io/routes-to-safety-workshop/demo/

Data bundle .../releases/tag/data-v2 (143 MB; scripts/get-bundle.sh)

Learn more duckdb.org/docs/extensions/spatial · docs.overturemaps.org · github.com/onthegomap/planetiler · docs.protomaps.com/pmtiles · maplibre.org · project-osrm.org/docs

Attribution to keep on every map: © OpenStreetMap contributors · © Overture Maps Foundation · Fire Hazard Severity Zones: CAL FIRE FRAP (CC-BY) · Land cover: ESA WorldCover (CC-BY) · Base map: © Esri, HERE, Garmin, USGS

Feedback form: [URL / QR to add]

Thank you.

Vishal Kumar · Shaishav Maisuria — questions: stay, or find us in the hallway. Teaching example, not an official evacuation resource. Drive-time estimates assume no traffic and no closures.

Routes to Safety · FOSS4G NA 2026 · CC BY-SA 4.0